Advanced Cloud Computing Revision Notes

14 Topics — From Mobile Cloud to Cyber-Physical Systems

Part 1 — Mobile Cloud Computing

Mobile Cloud Computing · Offloading using Cloudlet · MuSIC Framework · Mobile App Modelling

Part 2 — Geospatial & Data Centre

Geospatial Information · Geospatial Cloud · DC Architecture in Cloud

Part 3 — IoT, Sensors & Federation

WSN and IoT · Cloud Federation Architecture · Federation Types (Tightly/Loosely/Partially Coupled)
Federation Architectures (Multi-tier, Aggregated, Broker, Hybrid/Bursting)

Part 4 — Migration, Containers & Modern Paradigms

Pre-copy vs Post-copy Migration · Containers & Volumes with MERN Stack
Cyber-Physical Systems · Serverless Cloud · Sustainable Cloud Computing

April 25, 2026 • Beginner-Friendly with Stories, Analogies & Exam Definitions

Contents

1 Part 1 — Mobile Cloud Computing **2**
1.1 Mobile Cloud Computing (MCC) . . . 2
1.1.1 How It Works — Simple Flow . . . 2
1.1.2 Real-Life Examples . . . 2
1.1.3 Key Features . . . 2
1.2 Offloading Using Cloudlet . . . 4
1.2.1 Why Cloudlets Exist — The Distance Problem . . . 4
1.2.2 How Offloading with Cloudlet Works . . . 4
1.3 MuSIC Framework and Mobile Application Modelling . . . 6
1.3.1 MuSIC — Context-Aware Offloading . . . 6
1.3.2 Why MuSIC Is Smarter Than Normal Offloading . . . 6
1.3.3 Mobile Application Modelling . . . 7
1.3.4 How Apps Are Modelled . . . 7

2 Part 2 — Geospatial & Data Centre **8**
2.1 Geospatial Information and Geospatial Cloud . . . 8
2.1.1 What is Geospatial Information? . . . 8
2.1.2 Types of Geospatial Data . . . 8
2.1.3 What is Geospatial Cloud? . . . 8
2.1.4 How Geospatial Cloud Works . . . 8
2.2 Data Centre (DC) Architecture in Cloud Computing . . . 10
2.2.1 The Three Core Layers of DC Architecture . . . 10
2.2.2 How the Three Layers Work Together . . . 10
2.2.3 Key Architecture Concepts . . . 11
2.2.4 Types of Data Centre Architectures . . . 11

3 Part 3 — IoT, Sensors & Cloud Federation **12**
3.1 Wireless Sensor Networks (WSN) and IoT . . . 12
3.1.1 Wireless Sensor Networks (WSN) . . . 12
3.1.2 Internet of Things (IoT) . . . 12
3.1.3 How WSN and IoT Work Together . . . 12
3.2 Cloud Federation and Its Architecture . . . 14
3.2.1 Key Components of Cloud Federation Architecture . . . 14
3.2.2 Benefits of Cloud Federation . . . 14
3.2.3 Challenges of Cloud Federation . . . 14
3.3 Types of Cloud Federation (Coupling Levels) . . . 15
3.3.1 Loosely Coupled Federation . . . 15
3.3.2 Partially Coupled Federation . . . 15
3.3.3 Tightly Coupled Federation . . . 15
3.4 Cloud Federation Architectures . . . 17
3.4.1 Multi-Tier Architecture . . . 17
3.4.2 Aggregated Architecture . . . 17
3.4.3 Broker Architecture . . . 17
3.4.4 Hybrid/Bursting Architecture . . . 17

4 Part 4 — Migration, Containers & Modern Paradigms **19**

4.1 Pre-copy vs Post-copy VM Migration . . . 19
4.1.1 Pre-copy Migration . . . 19
4.1.2 Post-copy Migration . . . 19
4.1.3 Comparison Table . . . 20
4.2 Containers and Volumes — MERN Stack Example . . . 21
4.2.1 Why Containers for MERN? . . . 21
4.2.2 The Three Containers in MERN . . . 21
4.2.3 Why Volume is Critical for MongoDB . . . 21
4.2.4 Complete Flow in the MERN Container Setup . . . 21
4.3 Cyber-Physical Systems (CPS) . . . 23
4.3.1 Key Components of CPS . . . 23
4.3.2 The CPS Feedback Loop . . . 23
4.3.3 Real-World CPS Examples . . . 23
4.3.4 CPS vs IoT . . . 24
4.4 Serverless Cloud Computing . . . 25
4.4.1 Traditional vs Serverless — The Key Difference . . . 25
4.4.2 How Serverless Works . . . 25
4.4.3 Real-Life Examples . . . 25
4.4.4 Popular Serverless Platforms . . . 26
4.4.5 Advantages and Disadvantages . . . 26
4.5 Sustainable Cloud Computing (Green Cloud) . . . 27
4.5.1 Why This Matters — The Scale of the Problem . . . 27
4.5.2 Techniques for Sustainable Cloud Computing . . . 27
4.5.3 Real Companies Leading in Green Cloud . . . 28
4.5.4 Serverless vs Sustainable — How They Complement Each Other . . . 28

5 Master Summary — All 14 Topics 31

Part 1 — Mobile Cloud Computing

1.1 Mobile Cloud Computing (MCC)

What is Mobile Cloud Computing?

Mobile Cloud Computing (MCC) is the combination of mobile devices and cloud computing. Your phone has limited battery, storage, and processing power. Instead of doing everything on the phone, **heavy tasks are offloaded to powerful cloud servers**. Your phone acts as a remote control; the cloud does the real work.

▶ How It Works — Simple Flow

1. You open an app on your phone
2. The app sends data to cloud servers
3. Cloud servers process it (fast and powerful)
4. Result is sent back to your phone
5. It feels like your phone did everything — but the cloud helped behind the scenes

▶ Real-Life Examples

Google Photos — Storage + AI in the Cloud

When you upload photos, they are stored in the cloud (not just your phone). You can search "dog" or "birthday" and AI finds the right images instantly. The AI facial recognition and image processing all happen on cloud servers — your phone just displays the result. Even if you lose your phone, your photos are safe.

Cloud Gaming — Xbox Cloud Gaming / GeForce NOW

You can play high-end console-quality games on a basic phone. The game runs on powerful cloud servers. Your phone only streams the video frames and sends your controller inputs back. No expensive gaming hardware needed on your side.

Voice Assistants — Siri / Google Assistant

You say: "What is the weather in Bhopal?" Your voice is captured by the phone and sent to cloud servers. AI processes your speech, understands the question, fetches weather data, and sends back the spoken answer. Your phone just records and plays.

▶ Key Features

- **Offloading** — heavy computation moved to cloud, saving phone battery and CPU
- **Storage support** — cloud holds data, freeing phone memory
- **Scalability** — cloud scales automatically for more users
- **Battery saving** — less processing on phone = longer battery life

Feature	Traditional Mobile	Mobile Cloud
Processing	On phone only	Shared with cloud
Storage	Phone memory limited	Virtually unlimited (cloud)
Battery impact	High (runs everything)	Lower (cloud handles heavy work)
Performance	Limited by hardware	Scales with cloud power

Real-Life Analogy

Your phone = a student asking questions. The cloud = a supercomputer teacher solving the hard problems and sending back the answers. The student gets credit but the teacher did the heavy lifting.

Key Tip / Memory Trick

MCC One-liner: Mobile Cloud Computing = phone sends task → cloud processes → result returns to phone. Phone does less, cloud does more.

1.2 Offloading Using Cloudlet

What is Offloading Using Cloudlet?

Offloading = moving heavy computation from your phone to a more powerful system.
Cloudlet = a small, powerful server located physically *very close* to you (inside a cafe, campus, mall).
Together: Instead of sending heavy tasks to a far-away cloud (causing delay), you send them to a nearby cloudlet for near-instant results.

▶ Why Cloudlets Exist — The Distance Problem

In normal mobile cloud computing, data travels to distant servers (perhaps thousands of kilometres away). This causes:

- **High latency** (delay) — round-trip time is 100–500ms
- **High internet usage** — uploading heavy data constantly
- **Poor performance for real-time apps** — AR/VR, face recognition, live games

Cloudlets solve this by being nearby — the round-trip becomes 5–20ms.

▶ How Offloading with Cloudlet Works

1. You run a heavy app on your phone (e.g., face recognition)
2. Phone detects the task needs more power
3. Task is sent to the nearby cloudlet (local server within the same building)
4. Cloudlet processes it in milliseconds
5. Result is returned to your phone instantly

Face Recognition in a Shopping Mall

You open a smart shopping app that identifies products by pointing your camera at them. Instead of sending the image to a cloud server overseas (causing a 400ms delay), your phone sends it to a cloudlet installed inside the mall's Wi-Fi infrastructure. The result comes back in under 15ms. The experience feels instant and smooth.

AR/VR Gaming

AR games need real-time response — any delay above 20ms breaks the immersion and makes the experience unpleasant. A cloudlet nearby processes the AR computations locally, keeping the latency below 10ms. The far cloud cannot achieve this.

Real-Life Analogy

- **Far Cloud** = Ordering food from a restaurant in another city — takes hours (high latency)
- **Cloudlet** = Ordering from the restaurant next door — arrives in minutes (low latency)

The cloudlet is the "restaurant next door" of cloud computing.

Key Points

Benefits: Low latency · Reduced internet usage · Better real-time performance · Battery saving
Limitations: Not available everywhere · Needs infrastructure investment · Local server security concerns

Exam Definition

Offloading using cloudlet is a technique in Mobile Cloud Computing where computational tasks are transferred from a mobile device to a nearby cloudlet (a small local server close to the user) to reduce latency and improve real-time performance, instead of sending tasks to a distant central cloud.

1.3 MuSIC Framework and Mobile Application Modelling

▶ MuSIC — Context-Aware Offloading

What is MuSIC?

MuSIC (Mobile computation offloading using Spatial Information Context) is an **intelligent decision-making framework** that decides *when*, *where*, and *how* a mobile app should offload tasks — using real-time context like location, network condition, battery level, and available nearby resources.

▶ Why MuSIC Is Smarter Than Normal Offloading

Without MuSIC, offloading is blind:

- App always sends to cloud regardless of network quality
- Battery level ignored
- Nearby cloudlets not considered

With MuSIC, offloading is intelligent:

- **Checks location** — Is there a nearby cloudlet available?
- **Checks network** — Is the connection fast enough for offloading?
- **Checks battery** — Is it worth spending energy sending data?
- **Checks device load** — Is the phone currently overloaded?
- **Makes optimal decision** — Offload to cloud, cloudlet, or run locally

AR Navigation App with MuSIC

You are using an AR navigation app in a busy city. MuSIC constantly monitors your context:

- You are near a shopping mall with a cloudlet → MuSIC offloads AR processing to cloudlet (fast!)
- You enter a tunnel with no signal → MuSIC runs processing locally on phone
- Battery drops below 15% → MuSIC reduces offloading to save energy
- You enter a cafe with strong Wi-Fi → MuSIC switches to cloud for higher quality processing

All of this happens automatically, invisibly, in real-time.

MuSIC Exam Definition

MuSIC is a context-aware offloading framework that uses spatial and environmental information (location, network, battery, nearby resources) to intelligently decide the optimal execution strategy for mobile applications — whether to run locally, offload to a nearby cloudlet, or use the distant cloud.

▶ Mobile Application Modelling

What is Mobile Application Modelling?

Mobile application modelling is the technique of **representing a mobile app as structured components or a task graph**, so the system can analyse which parts are computation-heavy and which parts should run locally or be offloaded.

▶ How Apps Are Modelled

An app is broken into logical parts (modules/components):

- **UI (User Interface)** — display, rendering, user interaction
- **Computation tasks** — heavy processing (face recognition, video encoding)
- **Data processing** — filtering, aggregating data
- **Network communication** — sending/receiving data

These components are represented as a **Task Graph (DAG — Directed Acyclic Graph)**:

- Each task = a **node** in the graph
- Dependency between tasks = an **edge** connecting nodes
- Tasks with high computation weight = candidates for offloading

Video Editing App Model

The app is broken into tasks:
- **Apply video filter** — very heavy computation → **offload to cloud**
- **Display video playback** — UI task, needs device screen → **run locally on phone**
- **Save to storage** — depends on network availability → **decide at runtime**

The modelling framework identifies these weights and dependencies, then the MuSIC system decides where each task runs.

Key Tip / Memory Trick

MuSIC + App Modelling together:
App Modelling identifies *which tasks are heavy* → MuSIC decides *where those tasks should run*
Result: Optimal performance + battery + speed for every situation.

Part 2 — Geospatial & Data Centre

2.1 Geospatial Information and Geospatial Cloud

▶ What is Geospatial Information?

Geospatial Information

Geospatial information is any data that is linked to a **specific location on Earth**. It combines "where" (location: latitude, longitude, address) with "what" (attributes: temperature, population, traffic, elevation).

▶ Types of Geospatial Data

- **Spatial Data** — Location itself: coordinates, shapes of roads/buildings, boundaries
- **Attribute Data** — Properties tied to that location: temperature in Bhopal, population density of Delhi, traffic on NH-12

Everyday examples of geospatial data:

- Your current location on Google Maps
- Swiggy/Zomato delivery tracking (restaurant + rider + you = 3 geospatial points)
- Weather forecast for a specific city
- GPS coordinates of a crashed vehicle for emergency services

Real-world applications:

- Navigation systems (Google Maps, Apple Maps)
- Disaster management (earthquake, flood zone mapping)
- Agriculture (soil quality mapping, crop health monitoring)
- Urban planning (city layout, infrastructure design)
- Ride-hailing (Uber, Ola — matching driver to passenger by location)

▶ What is Geospatial Cloud?

Geospatial Cloud

Geospatial Cloud = storing, processing, and analysing **geospatial data using cloud platforms**. Instead of keeping maps and location data on local devices, everything is handled on powerful cloud servers — enabling real-time, scalable, worldwide access.

▶ How Geospatial Cloud Works

1. Data collected from satellites, GPS devices, sensors, drones
2. Stored in cloud (Google Cloud, Amazon AWS, Microsoft Azure)
3. Processed using powerful parallel computing (analysing millions of data points)
4. Results delivered to users via apps and APIs in real-time

Google Maps — Perfect Geospatial Cloud Example

When you use Google Maps for navigation:
- Your GPS location is tracked (geospatial data point)
- Real-time traffic from millions of other users is analysed (cloud processing)
- The fastest route is calculated (cloud computation)

• Turn-by-turn directions are sent to your phone (cloud response)
None of this computation happens on your phone — it all happens in Google's geospatial cloud infrastructure.

Aspect	Geospatial Information	Geospatial Cloud
Meaning	Location-based data	Cloud system to manage that data
Focus	The data itself	Storage + processing + delivery
Example	GPS coordinates	Google Maps entire back-end
Scale	Single data point	Billions of data points simultaneously

Key Tip / Memory Trick

Geospatial Information = what the data is (location + attributes)
Geospatial Cloud = the cloud infrastructure that stores, processes, and serves that data at scale

2.2 Data Centre (DC) Architecture in Cloud Computing

What is a Data Centre?

A **data centre** is a large facility filled with servers, storage systems, and networking equipment. When you use Google Drive, Instagram, or Netflix, your data is stored in a data centre somewhere in the world — not on your device.
DC Architecture = how the hardware and software inside a data centre is organised and interconnected to deliver cloud services reliably and efficiently.

▶ The Three Core Layers of DC Architecture

Layer 1 — Compute Layer (The Brain)

This layer does all the actual processing:

- Physical servers (rows of powerful computers)
- Virtual Machines running on those servers
- Containers (Docker) running within VMs or directly

Example: When you run a web application on AWS, it runs inside this layer.

Layer 2 — Storage Layer (The Memory)

This layer stores all persistent data:

- **Block storage** — like virtual hard drives (fast, for databases)
- **File storage** — shared folders, network file systems
- **Object storage** — cloud buckets for images, videos, backups (S3, Google Cloud Storage)

Example: Your Google Photos are stored in the object storage layer.

Layer 3 — Network Layer (The Nervous System)

This layer connects everything and carries data:

- Switches and routers (directing data traffic)
- Load balancers (distributing requests evenly)
- Firewalls (security filtering)
- CDN endpoints (delivering content closer to users)

Example: When you load Netflix, the network layer routes your request to the correct server and streams video to you.

▶ How the Three Layers Work Together

Opening a Website

1. Your request arrives through the **Network Layer** (routed by switches and load balancer)
2. Reaches a server in the **Compute Layer** (a VM processes your request)
3. VM fetches data from the **Storage Layer** (database, images, etc.)
4. Response travels back through the **Network Layer** to your browser

All of this happens in under 100 milliseconds.

▶ Key Architecture Concepts

Concept	What It Does in the DC
Virtualisation	One physical server runs many VMs — better resource utilisation
Load Balancing	Distributes traffic evenly — no server gets overwhelmed
Redundancy	Every critical component has a backup — if one fails, another takes over
Scalability	Add more servers when demand grows without disruption
Fault Tolerance	System continues working even when components fail

▶ Types of Data Centre Architectures

- **Traditional DC:** Fixed hardware, rigid configuration, less flexible. Used in older enterprise setups.
- **Cloud DC (Modern):** Virtualised, highly scalable. Used by AWS, Azure, GCP.
- **Software-Defined DC (SDDC):** Everything (compute, storage, network) controlled by software. Fully automated management. Most advanced.

Part 3 — IoT, Sensors & Cloud Federation

3.1 Wireless Sensor Networks (WSN) and IoT

▶ Wireless Sensor Networks (WSN)

What is a WSN?

A **Wireless Sensor Network (WSN)** is a collection of small, battery-powered **sensor nodes** that sense data from the physical environment (temperature, humidity, motion, pressure), communicate wirelessly with each other, and send data to a central base station.

Each sensor node contains:

- **Sensor** — measures the physical property (temperature, light, vibration)
- **Microcontroller** — processes the raw sensor data
- **Wireless communication module** — sends data to nearby nodes or gateway
- **Power source** — small battery (must last months or years)

Smart Farm Example

Sensors placed throughout a farm measure: soil moisture, temperature, humidity, and sunlight. They communicate wirelessly to a gateway at the farm boundary. The gateway forwards data to a cloud platform. The farmer checks crop health and irrigation status on their phone from anywhere.

▶ Internet of Things (IoT)

What is IoT?

The **Internet of Things (IoT)** is the network of physical devices ("things") connected to the internet that can collect data, communicate with each other, and be controlled remotely. Everyday objects become "smart" by gaining connectivity and intelligence.

IoT examples:

- Smart bulbs controlled from your phone
- Smart watches tracking your heartbeat
- Smart AC that adjusts temperature automatically
- Industrial machines that report their own maintenance needs
- Smart city traffic lights that respond to real-time congestion

▶ How WSN and IoT Work Together

WSN and IoT are complementary:

- **WSN** = the data collection layer (sensors measuring the physical world)
- **IoT** = the connectivity and intelligence layer (processing data, taking action via internet)

Combined Architecture Flow:

1. **Sensor Layer (WSN)** — sensors collect raw physical data
2. **Network Layer** — data transmitted wirelessly to gateway
3. **Cloud/IoT Processing Layer** — cloud platform analyses data, makes decisions

4. **Application Layer** — user sees insights via mobile/web app and takes action

Aspect	WSN	IoT
Primary focus	Data sensing	Connectivity + intelligence
Network scope	Local wireless	Global (internet)
Function	Data collection	Data processing + action
Example	Farm soil sensors	Smart irrigation controller

Key Tip / Memory Trick

WSN = the eyes and ears of IoT. WSN collects data from the physical world. IoT uses that data over the internet to make smart decisions and trigger actions.

3.2 Cloud Federation and Its Architecture

What is Cloud Federation?

Cloud Federation = multiple independent cloud providers **connecting and cooperating** to share resources, handle each other's overflow, and provide services as if they were one unified system. Instead of being locked into one cloud, resources are pooled across multiple clouds.

IPL Streaming Spike

A streaming app normally runs on AWS. During the IPL Final, 50 million people tune in simultaneously. AWS alone cannot handle this spike. With cloud federation, extra workload automatically shifts to Google Cloud and Azure. Users experience no buffering. After the match, resources scale back. Users never knew multiple clouds were involved.

▶ Key Components of Cloud Federation Architecture

1. Cloud Providers (Member Clouds):
The individual clouds (AWS, Azure, GCP) that join the federation. Each has its own infrastructure but agrees to share resources under federation rules.

2. Federation Manager (Central Brain/Broker):
The most important component. Acts as coordinator:

- Decides which cloud handles which workload
- Manages resource allocation across clouds
- Monitors performance and cost
- Enforces SLA (Service Level Agreements)

3. Resource Pooling Layer:
Resources from all clouds (CPU from one, storage from another, network from another) are combined into one logical pool. Users see a unified resource pool.

4. SLA and Policy Management:
Defines rules that all participating clouds must follow: performance targets, security requirements, cost limits.

5. User/Client Layer:
Users request services. They don't know or care which physical cloud handles their request — the federation manager decides transparently.

▶ Benefits of Cloud Federation

- **Scalability** — when one cloud is full, another takes over seamlessly
- **Cost optimisation** — workloads routed to the cheapest available provider
- **Reliability** — if one cloud fails, others continue serving
- **No vendor lock-in** — not dependent on a single provider

▶ Challenges of Cloud Federation

- Different clouds use different APIs — interoperability is complex
- Security policies may not match between providers
- Data transfer between clouds can be slow or expensive

- Managing multiple providers adds orchestration complexity

3.3 Types of Cloud Federation (Coupling Levels)

▶ Loosely Coupled Federation

Loosely Coupled Federation

Clouds operate mostly **independently**. They interact with each other only when necessary (e.g., overflow situations). Very little coordination between them. Each cloud maintains full autonomy.

- High independence for each cloud provider
- Minimal sharing of information or control
- Easy to implement but less efficient
- Like freelancers occasionally collaborating — no one is in charge

▶ Partially Coupled Federation

Partially Coupled Federation

Clouds **share some information and resources** (like available capacity), but each still keeps significant control over its own infrastructure. A middle ground between loose and tight coupling.

- Moderate information sharing (e.g., resource availability reports)
- Limited trust between providers
- Like classmates who share some notes but not everything

▶ Tightly Coupled Federation

Tightly Coupled Federation

A **central system controls all participating clouds** with high coordination and data sharing. Works almost like a single unified cloud system. Highly efficient but complex to manage.

- Central management of all resources
- High efficiency and optimisation possible
- Strong policies and consistent security
- Like a company with multiple branches and one strict headquarters

Feature	Loosely Coupled	Tightly Coupled
Independence	Very high	Very low
Coordination	Minimal	Strong and constant
Complexity	Simple	Complex
Efficiency	Lower	Higher
Trust	Low (minimal sharing)	High (full sharing)

Key Tip / Memory Trick

Coupling memory trick:
Loosely = Independent freelancers Partially = Classmates sharing some notes
Tightly = One company, many branches

3.4 Cloud Federation Architectures

▶ Multi-Tier Architecture

Multi-Tier Architecture

The system is divided into **layers (tiers)**, each running on potentially different cloud providers. Each tier handles a different function of the application.

Typical tiers:

- **Presentation tier** — UI/frontend (React app on AWS)
- **Application tier** — Backend logic (Node.js API on Azure)
- **Data tier** — Database (MySQL on Google Cloud)

Benefit: Each layer can be optimised and scaled independently. Each can use the best cloud for that specific function.

▶ Aggregated Architecture

Aggregated Architecture

Multiple cloud providers' resources are **combined into one large virtual resource pool**. Users see and access this as a single unified cloud system, unaware of which underlying provider is serving them.

- Resources from AWS + Azure + GCP appear as one pool
- Better resource utilisation — idle resources in one cloud used by another
- Like combining multiple water tanks into one large, unified water supply

▶ Broker Architecture

Broker Architecture

A **broker (intelligent middleman)** sits between users and multiple cloud providers. When a user requests a service, the broker evaluates all available providers based on price, performance, SLA, and current availability — then directs the request to the best option.

- Broker acts as a marketplace or matchmaker
- Handles provider selection transparently
- User doesn't interact directly with any cloud
- Like a travel agent who books you the best available flight

▶ Hybrid/Bursting Architecture

Hybrid/Bursting Architecture

Normal workload runs on a **private cloud or on-premise infrastructure**. When demand spikes and local capacity is insufficient, the excess workload automatically **"bursts" into a public cloud**.

E-Commerce During Big Billion Days

A retail company runs its website on its own private cloud (on-premise servers). On normal days, this is sufficient. During a big sale event, traffic multiplies 100x. The system automatically bursts into AWS to handle the extra load. After the sale, traffic drops and the AWS resources are released. The company only pays for cloud during the burst.

Architecture	Core Idea
Multi-tier	Different layers on different clouds — frontend here, backend there
Aggregated	All clouds combined into one big resource pool
Broker	Smart middleman selects the best cloud for each request
Hybrid/Bursting	Use private cloud normally, burst to public cloud during peaks

Part 4 — Migration, Containers & Modern Paradigms

4.1 Pre-copy vs Post-copy VM Migration

What is Live VM Migration?

Live VM migration = moving a Virtual Machine from one physical server to another **while it is still running**, without shutting it down. Users experience no downtime. There are two main techniques: pre-copy and post-copy.

▶ **Pre-copy Migration**

Pre-copy Migration

Send most of the VM's memory data to the destination server **before** stopping the VM. The VM keeps running throughout. Only at the very end, a brief pause sends the last remaining changes.

How it works step by step:

1. VM is running normally on Server A
2. Memory pages are copied to Server B (destination) — VM is still running
3. While copying, some pages get modified by the running VM ("dirty pages")
4. Modified pages are re-copied to Server B
5. This continues until very few dirty pages remain (convergence)
6. VM is paused briefly
7. Final small set of changes sent to Server B
8. VM resumes on Server B — migration complete

Real-Life Analogy

You are moving houses but sending your luggage to the new house *in advance* while still living in the old house. Only at the very end do you make the final move. Minimal disruption to your daily life.

Advantages: Very small downtime (milliseconds), safe and reliable, no crash risk.
Disadvantages: Some data copied multiple times (bandwidth waste), slow if VM is very active (keeps generating dirty pages).

▶ **Post-copy Migration**

Post-copy Migration

Move the VM to the destination server immediately with only **essential minimum data**, then fetch the remaining memory **on demand** as the VM needs it.

How it works step by step:

1. VM is paused on Server A
2. Minimal essential state (CPU registers, minimal memory) sent to Server B
3. VM starts running on Server B immediately

4. When the VM tries to access memory not yet transferred, it generates a **page fault**
5. Missing page is fetched on-demand from Server A
6. Over time, all pages migrate; VM runs fully independently on Server B

Real-Life Analogy

You move to the new house immediately with just your essentials (phone, keys, some clothes). The rest of your belongings are fetched later when you actually need them. You're living in the new house faster, but need to call back for things.

Advantages: Faster initial migration, each memory page copied only once (no redundancy).
Disadvantages: Risk of VM crash if network fails mid-migration, performance may be sluggish initially while waiting for page fetches.

▶ **Comparison Table**

Feature	Pre-copy	Post-copy
When data moves	Before VM moves	After VM moves
Downtime	Very low	Slightly higher
Data transfer risk	Low	Higher (network failure = crash)
Efficiency	Lower (re-copies dirty pages)	Higher (each page once)
Initial performance	Stable immediately	May lag (waiting for pages)
Use when	Reliability critical	Speed critical, stable network

Key Tip / Memory Trick

Pre-copy = send luggage first, then move. **Post-copy** = move first, fetch luggage later.
Pre-copy = safer. Post-copy = faster but riskier.

4.2 Containers and Volumes — MERN Stack Example

Containers and Volumes in Context

A **container** is a lightweight isolated environment that runs an application with everything it needs (code, libraries, config). In a **MERN stack** (MongoDB, Express, React, Node.js), each component runs in its own container.
A **volume** is persistent storage that exists *outside* containers, so data survives even when containers are stopped or deleted.

▶ Why Containers for MERN?

Without containers, a MERN app running on different machines faces "it works on my laptop" problems — library versions differ, config differs. With containers, each service is isolated, consistent, and portable everywhere.

▶ The Three Containers in MERN

Container 1 — React Frontend

- Contains: React code + Node.js + npm dependencies
- Served via Nginx or development server
- Communicates with backend via HTTP API calls
- *Example flow:* User opens browser → React container serves the UI

Container 2 — Node.js + Express Backend

- Contains: Express server, API routes, business logic
- Handles HTTP requests from React frontend
- Connects to MongoDB container for data
- *Example flow:* User clicks Login → React sends POST to backend → backend checks MongoDB

Container 3 — MongoDB Database

- Contains: MongoDB server and database engine
- Stores all user data, posts, etc.
- **Uses a volume** to persist data beyond container lifecycle
- *Example flow:* Backend queries MongoDB → MongoDB reads from volume → returns data

▶ Why Volume is Critical for MongoDB

The Volume Problem — Without vs With

Without Volume:
User registers → data stored inside MongoDB container → container crashes or is restarted → **all user data is permanently lost**. Unacceptable for any real application.
With Volume:
User registers → data stored in MongoDB volume (external storage) → container crashes → data is safe on volume → new container starts and attaches to same volume → all data still there.

▶ Complete Flow in the MERN Container Setup

1. User opens browser, connects to **React container** (port 3000)

2. React sends API request to **Node/Express container** (port 5000)
3. Express queries **MongoDB container** (port 27017)
4. MongoDB reads/writes from **volume** (persistent disk storage)
5. Data returned: MongoDB → Express → React → User sees result

Real-Life Analogy

- **Container** = Restaurant kitchen (runs the app, processes requests)
- **Volume** = Walk-in fridge (stores ingredients/data permanently)

If the kitchen shuts down (container stops), food inside is lost. But the walk-in fridge (volume) keeps everything safe — new kitchen attaches to the same fridge and continues.

Exam Answer

Containers are lightweight isolated environments used to run each component of a MERN application separately (React frontend, Node.js backend, MongoDB database). Volumes provide persistent storage outside containers so that database data (like user accounts and posts) remains safe even when containers are stopped, restarted, or replaced. For example, MongoDB runs in a container but stores all data in a volume, ensuring no data loss across restarts.

4.3 Cyber-Physical Systems (CPS)

What is a Cyber-Physical System?

A **Cyber-Physical System (CPS)** is a system that **tightly integrates computation (software), networking, and physical processes**. Sensors collect data from the real world, software processes it and makes decisions, and actuators act on the physical world — creating a continuous intelligent feedback loop.

▶ Key Components of CPS

- **Sensors** — gather data from the physical world (temperature, speed, pressure, images)
- **Actuators** — perform physical actions based on decisions (brakes, valves, motors, alarms)
- **Control Algorithms** — software that processes sensor data and decides what the actuators should do
- **Communication Networks** — connect all components (Wi-Fi, 5G, wired)
- **Embedded Systems** — small computers running the control software inside the device

▶ The CPS Feedback Loop

Sense → Compute → Act → Repeat continuously

This loop happens in real-time, often thousands of times per second.

▶ Real-World CPS Examples

Autonomous (Self-Driving) Vehicle

Cameras and radar **sense** the road, other vehicles, pedestrians, and traffic signs. The onboard AI **computes** the safest path, speed, and steering angle. Motors, brakes, and steering **act** to navigate safely. This happens 1,000+ times per second. A single failure in this feedback loop could be fatal — hence CPS must be extremely reliable and fast.

Smart Grid (Electric Power System)

Smart meters across the city **sense** electricity consumption in real-time. Control software **computes** supply/demand balance and predicts shortfalls. Automated switches and transformers **act** to re-route power, balance loads, and prevent blackouts. The physical electrical grid and digital control system work as one integrated CPS.

Healthcare — Insulin Pump

A continuous glucose monitor **senses** blood sugar levels every 5 minutes. The embedded control algorithm **computes** how much insulin is needed. The pump **acts** by automatically delivering the precise insulin dose. This CPS works 24/7, even while the patient sleeps, preventing dangerous blood sugar swings.

▶ CPS vs IoT

Aspect	IoT	CPS
Primary focus	Connectivity of devices	Physical-digital integration + control
Feedback loop	Optional	Essential and continuous
Real-time requirement	Often relaxed	Usually strict (safety-critical)
Example	Smart bulb you control	Self-driving car reacting to road

Key Points

CPS is the foundation of: Smart cities · Autonomous vehicles · Industrial automation (Industry 4.0) · Advanced healthcare · Smart grids
Key challenge: Security (cyberattacks can cause physical damage), real-time reliability, complex system design.

4.4 Serverless Cloud Computing

What is Serverless Computing?

Serverless computing does *not* mean there are no servers. It means **you do not need to manage any servers**. The cloud provider handles all server setup, scaling, maintenance, and infrastructure. You only write and upload small pieces of code (called **functions**), which run automatically in response to events.

▶ Traditional vs Serverless — The Key Difference

Traditional approach:

- Set up servers (or VMs)
- Install OS, runtime, libraries
- Configure scaling rules
- Monitor and maintain 24/7
- Pay whether the server is busy or idle

Serverless approach:

- Write your function (just the code that does the task)
- Upload it to the cloud
- It runs automatically when triggered
- Scales automatically from 0 to millions of calls
- Pay only for actual execution time (milliseconds)

▶ How Serverless Works

1. Developer writes a function (small piece of code)
2. Function is deployed to cloud provider (AWS Lambda, Google Cloud Functions, Azure Functions)
3. A trigger event occurs (user uploads a file, HTTP request arrives, database changes)
4. Cloud provider **automatically starts a container** for the function
5. Function executes (milliseconds to seconds)
6. Container shuts down — resources released
7. You are billed only for the execution time

▶ Real-Life Examples

CampusSync Student Assignment Uploader

A student uploads an assignment to the CampusSync app. This event triggers three serverless functions:

- **Function 1:** Stores the file in cloud storage (runs for 200ms)
- **Function 2:** Sends a notification email to the teacher (runs for 150ms)
- **Function 3:** Updates the student's progress dashboard (runs for 100ms)

No server runs 24/7 waiting for uploads. Functions activate only when an upload happens. Total billing: 450ms of compute time.

Image Processing Service

An e-commerce platform receives product photos uploaded by sellers. A serverless function automatically resizes each image to multiple resolutions (thumbnail, medium, large) and saves them to storage. The function runs only when a photo arrives. During sale season with thousands of uploads, it automatically scales to run thousands of parallel instances.

▶ Popular Serverless Platforms

- **AWS Lambda** — most widely used, triggered by S3, API Gateway, DynamoDB, etc.
- **Google Cloud Functions** — integrates with Firebase, Pub/Sub, HTTP
- **Azure Functions** — integrates with Azure ecosystem, Visual Studio

▶ Advantages and Disadvantages

Advantages	Details
No server management	Focus 100% on code, not infrastructure
Auto-scaling	Handles 10 or 10 million calls automatically
Pay-per-use	Pay only for actual execution (milliseconds)
Faster development	No setup time; deploy code instantly
Limitations	**Details**
Cold start latency	First invocation may take 100–500ms to start
Execution time limits	AWS Lambda max 15 minutes per invocation
State management	Functions are stateless; must use external storage
Vendor dependency	Closely tied to one provider's ecosystem

Key Tip / Memory Trick

Serverless = Don't worry about servers. Just write code and events trigger it.
Best for: event-driven tasks, microservices, APIs with variable traffic, background jobs.
Not ideal for: long-running processes, real-time systems needing ultra-low latency.

4.5 Sustainable Cloud Computing (Green Cloud)

What is Sustainable Cloud Computing?

Sustainable cloud computing (also called **Green Cloud**) focuses on designing and operating cloud systems to **minimise energy consumption, reduce carbon emissions, and use renewable energy sources**, while still delivering excellent performance. The goal is to make cloud computing environmentally responsible.

▶ Why This Matters — The Scale of the Problem

Global data centres consume approximately **1–2% of the world's total electricity**. A single large data centre uses as much electricity as a small city. Cloud computing is growing rapidly — without sustainable practices, the carbon footprint would grow proportionally.

The Idle Server Problem

A company runs 100 servers. On average, servers are used at only 20–30% capacity. The other 70–80% of energy is wasted on idle or lightly-loaded servers that nobody is turning off. This is like leaving 100 light bulbs on in a building where only 20 rooms are occupied.
Sustainable cloud: Use virtualisation to pack workloads onto 30 efficient servers. Power off the 70 idle servers. Same work done, 70% less electricity consumed.

▶ Techniques for Sustainable Cloud Computing

1. Server Consolidation + Virtualisation:
Pack multiple virtualised workloads onto fewer physical servers at high utilisation. Power off empty servers. Most impactful technique.

2. Renewable Energy:
Power data centres using solar panels, wind turbines, hydroelectric power instead of coal or gas. Google, Microsoft, and Amazon all have renewable energy commitments.

3. Energy-Efficient Hardware:
Use modern, energy-efficient processors and storage. SSDs use far less power than traditional hard drives. New CPUs deliver more performance per watt.

4. Smart Cooling Systems:
Cooling accounts for 30–40% of a data centre's energy use. Free-air cooling (using outside cold air), liquid cooling, and AI-driven temperature management reduce this dramatically. Google uses DeepMind AI to cut cooling energy by 40%.

5. Carbon-Aware Workload Scheduling:
Schedule non-urgent workloads (batch processing, backups) to run when the electrical grid is cleanest (more renewable energy available) and pause during dirty hours. Google does this for YouTube transcoding.

6. Dynamic Power Management (DVFS):
Reduce CPU clock speed and voltage when load is low. Even a 10% reduction in clock speed saves significant power in a large data centre.

▶ Real Companies Leading in Green Cloud

Company	Sustainability Initiative
Google	100% renewable energy since 2017; AI-optimised cooling; carbon-neutral operations
Microsoft	Aims to be carbon *negative* by 2030; underwater data centres for natural cooling
Amazon AWS	100% renewable energy goal; largest corporate buyer of renewable energy
Apple	All data centres run on 100% renewable energy since 2014

▶ Serverless vs Sustainable — How They Complement Each Other

Feature	Serverless	Sustainable Cloud
Primary focus	No server management	Energy efficiency
Goal	Simplicity + scalability	Eco-friendly computing
Key mechanism	Pay-per-use functions	Consolidation + renewables
Example	AWS Lambda	Google's AI-driven cooling
Connection	Serverless inherently reduces idle servers → supports sustainability goals	

Exam Definitions

Serverless Cloud Computing: A cloud model where developers write and deploy code as functions without managing any server infrastructure. The provider handles scaling, availability, and maintenance. Functions execute on demand and users pay only for actual execution time.

Sustainable Cloud Computing: The practice of designing and operating cloud infrastructure to minimise energy consumption and carbon emissions through server consolidation, renewable energy, efficient cooling, and smart workload scheduling, while maintaining performance and reliability.

Key Tip / Memory Trick

Serverless = Don't worry about servers.
Sustainable = Don't waste energy.
Both together = A cloud that is easy to use AND kind to the planet.

Master Summary — All 14 Topics

#	Topic	Core Concept	Key Example
1	Mobile Cloud Computing	Phone + cloud work together	Google Photos, cloud gaming
2	Offloading via Cloudlet	Send tasks to nearby mini-cloud	Face recognition in mall
3	MuSIC Framework	Smart context-aware offloading	AR app adapts to location
4	Mobile App Modelling	Task graph for offload decisions	DAG of video editor tasks
5	Geospatial Information	Location-linked data	GPS coordinates, traffic
6	Geospatial Cloud	Cloud stores and processes location data	Google Maps backend
7	DC Architecture	3 layers: Compute, Storage, Network	Netflix streaming system
8	WSN and IoT	Sensors collect, IoT connects and acts	Smart farm irrigation
9	Cloud Federation	Multiple clouds work as one	IPL streaming spike handling
10	Federation Coupling	Loose/Partial/Tight coupling levels	Freelancers to HQ branches
11	Federation Architectures	Multi-tier, Aggregated, Broker, Hybrid	Travel agent = Broker
12	Pre/Post-copy Migration	Move VM while running	Luggage before vs after move
13	Containers + Volumes (MERN)	Each service in container; data in volume	MongoDB volume persists data
14	Cyber-Physical Systems	Digital-physical feedback loop	Self-driving car, smart grid
15	Serverless Computing	No server management, functions on demand	AWS Lambda, Cloud Functions
16	Sustainable Cloud	Energy efficiency + renewables	Google AI cooling, solar DCs

Final Golden Rules — All 14 Topics

MCC: Phone + cloud = together. Phone does less, cloud does more.

Cloudlet: Far cloud = slow (400ms). Nearby cloudlet = fast (10ms). Use for real-time apps.

MuSIC: Smart offloading using location + network + battery + available resources.

Geospatial: Location data + cloud = real-time scalable geographic services.

DC Architecture: Compute (brain) + Storage (memory) + Network (nervous system).

WSN + IoT: WSN = eyes/ears (sense). IoT = brain/hands (connect + act).

Federation: Loose = independent. Partial = some sharing. Tight = one control. Broker = middleman.

Hybrid/Bursting: Private cloud normally; public cloud during peaks.

Pre-copy: Move luggage first. **Post-copy:** Move first, get luggage later.

MERN Containers: React + Node + MongoDB in separate containers. Volume saves MongoDB data.

CPS: Sense → Compute → Act → Repeat. Physical + digital = one system.

Serverless: Write function, cloud runs it on trigger, pay per millisecond.

Sustainable: Consolidate servers, use renewables, AI cooling, carbon-aware scheduling.